

SQL INTERVIEW QUESTION BANK

Stock Portfolio & Investment Tracker Project

35 interview questions across 5 categories — schema design, query logic, window functions, error handling, and conceptual understanding. Each question is tagged by difficulty with a model answer and interview tip.

Total Questions	35
Difficulty Levels	Easy Medium Hard
Categories	5
Target Role	Data Analyst Intern / Junior Data Analyst
Date	March 22, 2026

How to Use This Guide

These questions are drawn directly from the Stock Portfolio & Investment Tracker project. They reflect the kinds of questions asked in data analyst interviews at fintech companies, investment banks, and analytics-focused startups.

EASY	Foundational concepts — expect these in every SQL round
MEDIUM	Requires reasoning — the majority of interview questions fall here
HARD	Advanced patterns — differentiates you from other candidates

Category 1 — Schema Design & Data Modelling

These questions test whether you understand why the database is structured the way it is — not just what the tables contain.

Q1

Why does the trades table have both portfolio_id and stock_id as foreign keys?

EASY

A trade record is meaningless without knowing which portfolio executed it and which stock was traded. Removing portfolio_id means you cannot associate the trade with an investor. Removing stock_id means you cannot identify what was bought or sold. Both foreign keys are required for the row to have any analytical value. This is a fundamental principle of relational design — child tables hold foreign keys because they need parent context to give their own data meaning.

***Interview Tip:** Interviewers ask this to test if you understand FK placement beyond memorisation. Lead with 'the row becomes meaningless without it.'*

Q2

Why is trade_type defined as ENUM('BUY','SELL') rather than VARCHAR?

EASY

ENUM restricts the column to a predefined set of valid values, enforcing data integrity at the database level. VARCHAR accepts any string — including typos like 'buy', 'Buy', 'BOUGHT', or accidental spaces — which would cause GROUP BY and WHERE filters to silently return incorrect results. For categorical columns with a fixed, known set of values, ENUM is always the correct choice.

***Interview Tip:** Follow up with: 'This kind of constraint belongs at the database layer, not the application layer, because applications can change but the data must always be consistent.'*

Q3

Why is there a UNIQUE constraint on daily_prices(stock_id, price_date)?

EASY

daily_prices should have exactly one record per stock per trading day. Without the UNIQUE constraint, duplicate records could be inserted — for example if a data pipeline runs twice. Duplicate price records would cause moving averages and volume calculations to produce inflated, incorrect results. The constraint prevents this at the database level without requiring application-side duplicate checking.

***Interview Tip:** This shows you think about data quality proactively, not just query correctness. Mention it as a data integrity decision.*

Q4 What is a star schema and how does this project follow that pattern?**MEDIUM**

A star schema organises tables into fact tables (high-volume transactional records) and dimension tables (descriptive context). In this project, trades and daily_prices are fact tables — they contain the numeric events (each trade, each price record). investors, portfolios, and stocks are dimension tables — they provide context about who, what, and where. The fact tables reference dimension tables via foreign keys, forming a star-shaped relationship diagram. This is the standard structure used in financial databases and data warehouses.

***Interview Tip:** Saying 'star schema' in an interview at a fintech or analytics company immediately signals you understand data modelling beyond basic normalisation.*

Q5 Why do date-only fields use DATE instead of DATETIME?**EASY**

DATETIME stores both date and time (e.g. 2024-01-15 14:32:00). For fields like trade_date, ex_date, and price_date, the time component is irrelevant and storing it wastes space. More importantly, date comparisons become error-prone — a WHERE trade_date = '2024-01-15' filter will fail to match a DATETIME value of '2024-01-15 09:00:00' without explicit casting. DATE type keeps comparisons clean and storage efficient.

***Interview Tip:** This is a common mistake in junior developer schemas. Knowing it proactively shows attention to detail.*

Q6 Why are indexes created on trade_date, portfolio_id, and price_date?**MEDIUM**

trades and daily_prices are the largest tables in the schema — trades grows with every transaction, daily_prices accumulates thousands of rows per year across 15 stocks. Without indexes, every query that filters or joins on these columns performs a full table scan — reading every row regardless of relevance. Indexes create a sorted lookup structure that allows MySQL to jump directly to matching rows, dramatically reducing query execution time on large datasets.

***Interview Tip:** Indexes are a favourite interview topic. Follow up: 'I wouldn't index every column — indexes slow down INSERT and UPDATE operations, so they're worth it only on columns used frequently in WHERE, JOIN, or ORDER BY clauses.'*

Category 2 — Query Logic & SQL Fundamentals

Q7

What is the difference between **WHERE** and **HAVING**? Give an example from this project.

EASY

WHERE filters individual rows before **GROUP BY** executes — aggregate functions like **SUM()** and **COUNT()** do not exist yet at that stage. **HAVING** filters after **GROUP BY**, operating on aggregated results. In this project, the query to find portfolios above the platform average uses **HAVING** because the comparison is against **SUM(quantity * price_per_share)** — an aggregate that only exists after grouping.

```
-- WHERE: filters rows before grouping
SELECT * FROM trades WHERE trade_type = 'BUY';

-- HAVING: filters after grouping
SELECT portfolio_id, SUM(quantity * price_per_share) AS total
FROM trades
GROUP BY portfolio_id
HAVING total > 50000;
```

Interview Tip: This is asked in almost every SQL interview. The one-line answer: 'WHERE filters rows, HAVING filters groups.'

Q8

Why can't you use a column alias defined in **SELECT** inside the **WHERE** clause?

MEDIUM

SQL execution order is: **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY**. The **SELECT** clause — where aliases are defined — runs after **WHERE**. So when **WHERE** executes, the alias does not yet exist. This is why you must repeat the full expression in **WHERE** or use a subquery/CTE to pre-compute the value. Aliases defined in **SELECT** are available in **ORDER BY** because **ORDER BY** runs last.

Interview Tip: Understanding execution order is a strong signal of SQL depth. Interviewers often ask 'why does this query fail?' — execution order explains most such edge cases.

Q9 How do you calculate current net share holdings per portfolio per stock?**MEDIUM**

BUY trades add shares, SELL trades remove them. Net position = total bought - total sold. This is implemented using conditional aggregation inside SUM() — treating BUY quantity as positive and SELL quantity as negative. HAVING filters out fully exited positions (net = 0).

```
SELECT portfolio_id, stock_id,
SUM(CASE WHEN trade_type = 'BUY' THEN quantity
WHEN trade_type = 'SELL' THEN -quantity END) AS shares_held
FROM trades
GROUP BY portfolio_id, stock_id
HAVING shares_held > 0;
```

Interview Tip: This pattern — SUM(CASE WHEN) with positive/negative values — appears frequently in finance SQL. It is more efficient than two separate subqueries.

Q10 What is the difference between COUNT(*), COUNT(column), and COUNT(CASE WHEN)?**MEDIUM**

COUNT(*) counts every row including NULLs. COUNT(column) counts non-NULL values in that column. COUNT(CASE WHEN condition THEN 1 END) counts only rows where the condition is true — because CASE WHEN returns NULL when the condition is false, and COUNT ignores NULLs. This last pattern is used in this project to split BUY and SELL trade counts into separate columns without running two separate queries.

Interview Tip: Interviewers love this question. The key insight is that COUNT ignores NULLs — that is what makes COUNT(CASE WHEN) work.

Q11 Why is LEFT JOIN used in some queries and INNER JOIN in others?**MEDIUM**

INNER JOIN returns only rows with a match in both tables — unmatched rows are dropped silently. LEFT JOIN returns all rows from the left table, with NULLs for columns from the right table when there is no match. In this project, LEFT JOIN is used when some portfolios may not have trades yet — dropping them with INNER JOIN would give incomplete results. INNER JOIN is used in the dividend holdings CTE where a match must exist (a stock must be in trades to be relevant).

Interview Tip: The word 'silently' matters here. INNER JOIN does not throw an error when rows are dropped — it just removes them. Unintended INNER JOINs are a common source of incorrect results that are hard to debug.

Q1 2 Why do you need to JOIN on both portfolio_id AND stock_id when joining the avg_buy_cost CTE?**HARD**

The same stock can exist in multiple portfolios, each with a different average buy price. If you join only on stock_id, the average buy price from portfolio A would incorrectly match against SELL trades from portfolio B. Both columns are needed to uniquely identify the correct cost basis for each (portfolio, stock) combination.

```
JOIN avg_buy_cost abc
ON abc.portfolio_id = t.portfolio_id
AND abc.stock_id = t.stock_id
```

Interview Tip: This is a subtle but important point. Mention it proactively when discussing the P&L; query — it shows you thought carefully about data correctness, not just syntax.

Q1 3 What is the difference between a subquery and a CTE? When would you choose each?**MEDIUM**

Both are temporary result sets that exist only during query execution. A subquery is embedded inline and has no name — it cannot be referenced more than once. A CTE is defined before the main query with a name using the WITH clause — it can be referenced multiple times and is far more readable for complex logic. Use subqueries for simple, one-time comparisons (like HAVING total > (SELECT AVG(...))). Use CTEs when the intermediate result is needed in multiple places or when the logic has multiple steps (like the P&L; query).

Interview Tip: A concrete rule: if you find yourself copy-pasting the same subquery twice, replace it with a CTE.

Q1 4 What does COALESCE do and why is it used in the dividend income query?**MEDIUM**

COALESCE returns the first non-NULL value from a list of arguments. In the dividend query, investors who only bought and never sold a stock will have NULL in the SELL side of the SUM(CASE WHEN) calculation. Without COALESCE, NULL propagates through arithmetic — any expression involving NULL returns NULL. COALESCE(value, 0) replaces NULL with 0, ensuring the net shares calculation produces a valid number.

Interview Tip: A related pattern: ISNULL() in SQL Server, NVL() in Oracle. In MySQL, COALESCE is the standard choice and works across all databases.

Category 3 — Window Functions

Window functions are the most commonly tested advanced SQL topic in data analyst interviews. These questions are based directly on the Tier 3 queries in this project.

Q1
5

What is a window function and how is it different from GROUP BY?

EASY

GROUP BY collapses multiple rows into one row per group, losing individual row detail. A window function computes across a group of rows while keeping every individual row visible in the output. The OVER() clause defines the window — PARTITION BY sets the group scope (like GROUP BY) and ORDER BY sets the row sequence within that group. Unlike GROUP BY, window functions never reduce the number of rows in the result.

Interview Tip: Use the exam marksheet analogy: GROUP BY gives you the class average (one row per class). A window function gives you each student's score AND the class average on the same row.

Q1
6

What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

MEDIUM

All three assign a sequential number to rows within a partition. They differ in tie handling: RANK() gives tied rows the same number and skips the next rank (1, 1, 3). DENSE_RANK() gives tied rows the same number but does not skip (1, 1, 2). ROW_NUMBER() always assigns a unique number regardless of ties (1, 2, 3). In this project, RANK() is used for sector stock rankings — two stocks with equal volume receive the same rank, and the next rank is skipped.

```
RANK() OVER (PARTITION BY sector ORDER BY volume DESC) -- 1,1,3
DENSE_RANK() OVER (PARTITION BY sector ORDER BY volume DESC) -- 1,1,2
ROW_NUMBER() OVER (PARTITION BY sector ORDER BY volume DESC) -- 1,2,3
```

Interview Tip: 'Find the second highest salary per department' is the most common SQL interview question. DENSE_RANK() is the correct tool — RANK() would skip rank 2 if there are ties at rank 1.

Q1
7

How does PARTITION BY differ from GROUP BY?

MEDIUM

GROUP BY collapses rows into one row per group. PARTITION BY inside a window function divides rows into groups for the window calculation but does not collapse them — all rows remain visible. PARTITION BY is essentially GROUP BY without the row reduction. In the running total query, PARTITION BY portfolio_id means the cumulative sum resets to zero at the start of each new portfolio, while all individual trade rows remain in the output.

Interview Tip: This distinction is one of the most frequently asked follow-up questions after any window function answer.

Q1
8**Explain the ROWS BETWEEN clause used in the 7-day moving average query.**

HARD

ROWS BETWEEN defines the frame — the subset of rows within the window partition that the function operates on for each row. ROWS BETWEEN 6 PRECEDING AND CURRENT ROW means: for each row, include the current row and the 6 rows before it, giving a 7-row window. When fewer than 7 rows exist (at the start of the price history), MySQL averages whatever rows are available without throwing an error. This produces the smoothed moving average used in trading dashboards.

```
AVG(close_price) OVER (  
  PARTITION BY stock_id  
  ORDER BY price_date  
  ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
) AS moving_avg_7day
```

Interview Tip: Common interview extension: 'What is the difference between ROWS BETWEEN and RANGE BETWEEN?' ROWS is based on physical row position. RANGE is based on the value of the ORDER BY column — ties are treated as the same row.

Q1
9**What does NTILE(4) do and what is it used for in this project?**

MEDIUM

NTILE(n) divides rows into n equal-sized buckets and assigns each row a bucket number from 1 to n. NTILE(4) creates quartiles — the bottom 25% of rows get bucket 1, the top 25% get bucket 4. In this project it is used to segment investors into four tiers — Standard, Silver, Gold, Platinum — based on total investment amount. This is a standard customer segmentation technique used in wealth management platforms.

Interview Tip: NTILE is less commonly known than RANK — mentioning it unprompted demonstrates breadth of window function knowledge.

Q2
0**What is the default window frame when ORDER BY is specified but ROWS BETWEEN is not?**

HARD

When ORDER BY is present inside OVER() without an explicit ROWS BETWEEN clause, MySQL defaults the frame to RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW — equivalent to all rows from the start of the partition up to the current row. This is usually the correct behaviour for cumulative sums. However for LAST_VALUE(), this default frame means the 'last' value is always the current row — which produces incorrect results. LAST_VALUE() requires an explicit ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING to work correctly.

Interview Tip: The LAST_VALUE default frame gotcha is a classic trick question. Most candidates who know LAST_VALUE get this wrong. Knowing it signals you have used window functions in practice, not just read about them.

Q2
1**How would you calculate the difference between each day's closing price and the previous day's?****MEDIUM**

Use the LAG() window function, which accesses the value of a column from a previous row within the partition. LAG(close_price, 1) returns the close price from 1 row earlier. Subtracting it from the current close_price gives the daily price change. The first row in each partition returns NULL because there is no preceding row.

```
SELECT ticker, price_date, close_price,  
       close_price - LAG(close_price, 1)  
       OVER (PARTITION BY stock_id ORDER BY price_date)  
       AS daily_change  
FROM daily_prices dp  
JOIN stocks s ON s.stock_id = dp.stock_id;
```

Interview Tip: LAG and LEAD are frequently asked. LAG looks back, LEAD looks forward. Common use: 'compare this month's sales to last month's.'

Category 4 — Finance Domain Knowledge

At fintech and finance-adjacent companies, interviewers expect you to understand the domain vocabulary behind your queries — not just the SQL syntax.

Q2
2

What is cost basis and how is it calculated in the P&L; query?

MEDIUM

Cost basis is the average price paid per share for a position, used to calculate profit or loss when shares are sold. In this project it is calculated as $\text{SUM}(\text{quantity} * \text{price_per_share}) / \text{SUM}(\text{quantity})$ across all BUY trades for a given (portfolio, stock) combination. This weighted average correctly accounts for investors who made multiple purchases at different prices — simple averaging of prices would be mathematically incorrect.

```
-- Cost basis CTE
SELECT portfolio_id, stock_id,
SUM(quantity * price_per_share) / SUM(quantity) AS avg_buy_price
FROM trades
WHERE trade_type = 'BUY'
GROUP BY portfolio_id, stock_id;
```

Interview Tip: Interviewers at investment companies will test this. The wrong answer is $\text{AVG}(\text{price_per_share})$ — that ignores quantity weighting and gives an incorrect cost basis.

Q2
3

What is an ex-date and how does it affect dividend calculations?

MEDIUM

The ex-date (ex-dividend date) is the cutoff date for dividend eligibility. An investor must own shares before the ex-date to receive the dividend — purchasing on or after the ex-date means the investor does not qualify for that dividend payment. In the dividend income query, this is enforced by filtering trades with $\text{trade_date} < \text{ex_date}$ before calculating net shares held. The pay_date is when the dividend is actually transferred to the investor's account.

Interview Tip: Finance-domain SQL questions like this test whether you truly understand the data you are querying. Not knowing what ex_date means would be a red flag in a fintech interview.

Q2
4

What is realised vs unrealised P&L;?

MEDIUM

Realised P&L; is profit or loss from completed transactions — shares that have been bought and subsequently sold. This is what the P&L; query in this project calculates. Unrealised P&L; is the gain or loss on positions still held — calculated by comparing current market price against the cost basis. Unrealised P&L; can only be computed by joining current holdings against the latest daily_prices data. Tax is typically only triggered on realised P&L;.

Interview Tip: If you want to extend this project, writing a query for unrealised P&L; would be an excellent addition — it demonstrates you understand the distinction and can implement it.

Q2
5**What does a 7-day moving average tell you that a daily closing price does not?**

EASY

A daily closing price reflects a single day's market activity and contains a lot of noise — short-term fluctuations driven by intraday events, news, and random volatility. A moving average smooths these fluctuations by averaging prices over a rolling window, making the underlying trend more visible. Traders use moving averages to identify trend direction (price above the moving average = uptrend) and crossover signals. The 7-day window in this project is a short-term average — analysts also commonly use 50-day and 200-day moving averages.

Interview Tip: This question tests whether you understand why you wrote the query, not just how. Always be ready to explain the business purpose behind every query in your portfolio.

Q2
6**How would you modify the P&L; query to calculate unrealised P&L; for open positions?**

HARD

Join the current net holdings (from the position sizing query) against the latest price from daily_prices, then compare against the cost basis CTE. Unrealised P&L; = (latest close price - avg buy price) * shares currently held.

```
WITH avg_buy AS (  
  SELECT portfolio_id, stock_id,  
    SUM(quantity*price_per_share)/SUM(quantity) AS avg_buy_price  
  FROM trades WHERE trade_type='BUY'  
  GROUP BY portfolio_id, stock_id  
) ,  
holdings AS (  
  SELECT portfolio_id, stock_id,  
    SUM(CASE WHEN trade_type='BUY' THEN quantity  
      WHEN trade_type='SELL' THEN -quantity END) AS shares_held  
  FROM trades GROUP BY portfolio_id, stock_id  
  HAVING shares_held > 0  
) ,  
latest_price AS (  
  SELECT stock_id, close_price  
  FROM daily_prices  
  WHERE price_date = (SELECT MAX(price_date) FROM daily_prices)  
)  
SELECT h.portfolio_id, h.stock_id,  
  ROUND((lp.close_price - ab.avg_buy_price) * h.shares_held, 2) AS unrealised_pnl  
FROM holdings h  
JOIN avg_buy ab ON ab.portfolio_id=h.portfolio_id AND ab.stock_id=h.stock_id  
JOIN latest_price lp ON lp.stock_id=h.stock_id;
```

Interview Tip: This is an excellent extension to add to your GitHub project. It uses three chained CTEs and demonstrates real financial analysis capability.

Category 5 — Error Handling & Edge Cases

Q2
7**What causes MySQL Error 1055 and how do you fix it?****EASY**

Error 1055 (ONLY_FULL_GROUP_BY) occurs when a column in the SELECT clause is neither in the GROUP BY clause nor wrapped in an aggregate function. MySQL requires that every non-aggregated column in SELECT must be explicitly grouped. Fix: either add the column to GROUP BY, or wrap it in an aggregate like MAX() or ANY_VALUE() if grouping by it is not meaningful.

***Interview Tip:** This is one of the most common errors beginners encounter. Knowing the name 'ONLY_FULL_GROUP_BY' and the exact reason shows SQL fluency.*

Q2
8**What is Error 1052 and when does it appear?****EASY**

Error 1052 (Column is ambiguous) occurs when a column name exists in more than one joined table and the query does not specify which table's column to use. MySQL cannot resolve the ambiguity automatically and throws this error. Fix: prefix the column with the table name or alias — for example trades.portfolio_id instead of just portfolio_id.

***Interview Tip:** Always prefix columns with table aliases when writing multi-table queries. It eliminates ambiguity errors and makes queries significantly more readable.*

Q2
9**Why does MAX(SUM()) throw Error 1111?****MEDIUM**

Error 1111 (Invalid use of group function) occurs because MySQL does not allow nesting aggregate functions directly — you cannot wrap SUM() inside MAX() in the same SELECT clause. The correct approach is to compute SUM() first in a subquery or CTE, then apply MAX() or any other aggregate to the result in the outer query.

```
-- WRONG: nested aggregates
SELECT MAX(SUM(quantity)) FROM trades GROUP BY portfolio_id;

-- CORRECT: pre-aggregate in subquery
SELECT MAX(total) FROM (
  SELECT SUM(quantity) AS total
  FROM trades GROUP BY portfolio_id
) AS sub;
```

***Interview Tip:** Whenever you need to aggregate an aggregation, the solution is always a subquery or CTE first.*

Q3
0**What is Error 1066 and how does it occur in JOIN queries?**

MEDIUM

Error 1066 (Not unique table alias) occurs when the same table is referenced twice in a query without using different aliases to distinguish them. This often happens when a developer accidentally writes two LEFT JOIN statements for the same table. Fix: use table aliases to differentiate the references, or consolidate the conditions into a single JOIN using AND.

```
-- WRONG: same table joined twice
LEFT JOIN avg_buy_cost ON avg_buy_cost.stock_id = t.stock_id
LEFT JOIN avg_buy_cost ON avg_buy_cost.portfolio_id = t.portfolio_id

-- CORRECT: single JOIN with both conditions
LEFT JOIN avg_buy_cost
ON avg_buy_cost.stock_id = t.stock_id
AND avg_buy_cost.portfolio_id = t.portfolio_id
```

Interview Tip: The fix also improves query correctness — two separate JOINS on the same table produce a Cartesian product for the matched rows, not just an error.

Q3
1**What happens if you forget to add a SELECT after a CTE definition?**

EASY

MySQL throws Error 1064 (syntax error) because a CTE (WITH clause) is not a standalone statement — it is a prefix to a SELECT, INSERT, UPDATE, or DELETE query. Without the following SELECT, the syntax is incomplete. The WITH block defines a temporary named result set, but something must consume it. Always follow a CTE immediately with the main query.

Interview Tip: A quick way to debug a CTE during development: add `SELECT * FROM cte_name;` after the CTE to verify its output before building the full main query.

Q3
2**How do you handle NULL values that result from LEFT JOINs in calculations?**

MEDIUM

NULL propagates through arithmetic — any expression involving NULL returns NULL. When a LEFT JOIN produces NULLs (because no matching row exists in the right table), calculations like SUM() or subtraction will return NULL instead of a number. Use COALESCE(value, 0) to replace NULL with a default value, or use INNER JOIN if rows without a match should be excluded entirely.

Interview Tip: 'NULL is not zero' is a fundamental SQL concept. Interviewers often present a query with unexpected NULLs in the output and ask you to explain and fix it.

Q3 3 A portfolio has SELL trades but no BUY trades in the dataset. What happens in the P&L; query?**HARD**

The avg_buy_cost CTE filters WHERE trade_type = 'BUY', so this portfolio will have no matching row in the CTE. When the main query joins SELL trades against the CTE, the LEFT JOIN returns NULL for avg_buy_price. The P&L; calculation then produces NULL. This is addressed by filtering WHERE avg_buy_price IS NOT NULL in the main query — or by using INNER JOIN to automatically exclude unmatched SELL trades. In a real system, this data quality issue should be investigated at the source.

***Interview Tip:** This is the kind of edge case thinking that separates strong candidates. Always ask: 'what happens when the data is incomplete or inconsistent?'*

Q3 4 What is the risk of not using HAVING shares_held > 0 in the net position query?**MEDIUM**

Without HAVING shares_held > 0, the query returns positions where the investor has fully exited (net shares = 0) and even positions where SELL quantity exceeds BUY quantity (negative holdings — which would indicate a short position or a data error). Including these rows in downstream analysis — like unrealised P&L; or dividend calculations — would produce incorrect results. The HAVING clause ensures only active, positive holdings are reported.

***Interview Tip:** This shows you think about what the data means, not just whether the query runs. Interviewers value this kind of data-aware reasoning.*

Q3 5 How would you find investors who have never made a SELL trade?**HARD**

Use NOT EXISTS with a correlated subquery — it checks whether a related row exists without actually joining. This is more reliable than NOT IN when NULLs might be present in the subquery result, as NOT IN with a NULL value always returns an empty result.

```
SELECT DISTINCT i.full_name
FROM investors i
JOIN portfolios p ON p.investor_id = i.investor_id
WHERE NOT EXISTS (
  SELECT 1 FROM trades t
  WHERE t.portfolio_id = p.portfolio_id
  AND t.trade_type = 'SELL'
);
```

***Interview Tip:** NOT EXISTS vs NOT IN is a classic interview question. The answer: NOT IN fails silently when NULLs are present in the subquery. NOT EXISTS handles NULLs correctly and is generally the safer, more readable choice.*

Quick Reference — Questions by Difficulty

#	Question Summary	Category	Difficulty
1	Why trades table has two FKs	Schema	Easy
2	ENUM vs VARCHAR for trade_type	Schema	Easy
3	UNIQUE constraint on daily_prices	Schema	Easy
4	Star schema pattern	Schema	Medium
5	DATE vs DATETIME	Schema	Easy
6	Why indexes are created	Schema	Medium
7	WHERE vs HAVING	Query Logic	Easy
8	Column alias in WHERE clause	Query Logic	Medium
9	Net share holdings calculation	Query Logic	Medium
10	COUNT(*) vs COUNT(col) vs COUNT(CASE WHEN)	Query Logic	Medium
11	LEFT JOIN vs INNER JOIN	Query Logic	Medium
12	JOIN on two columns in CTE	Query Logic	Hard
13	Subquery vs CTE	Query Logic	Medium
14	COALESCE usage	Query Logic	Medium
15	Window function vs GROUP BY	Window Fn	Easy
16	RANK vs DENSE_RANK vs ROW_NUMBER	Window Fn	Medium
17	PARTITION BY vs GROUP BY	Window Fn	Medium
18	ROWS BETWEEN clause	Window Fn	Hard
19	NTILE(4) and customer segmentation	Window Fn	Medium
20	Default window frame gotcha	Window Fn	Hard
21	LAG() for daily price change	Window Fn	Medium
22	Cost basis calculation	Finance	Medium
23	Ex-date and dividend eligibility	Finance	Medium
24	Realised vs unrealised P&L;	Finance	Medium
25	Moving average vs daily price	Finance	Easy
26	Unrealised P&L; extension query	Finance	Hard
27	Error 1055 ONLY_FULL_GROUP_BY	Error Handling	Easy
28	Error 1052 ambiguous column	Error Handling	Easy

29	Error 1111 nested aggregates	Error Handling	Medium
30	Error 1066 duplicate table alias	Error Handling	Medium
31	CTE without SELECT	Error Handling	Easy
32	NULL in LEFT JOIN calculations	Error Handling	Medium
33	SELL with no matching BUY	Error Handling	Hard
34	Risk of missing HAVING clause	Error Handling	Medium
35	NOT EXISTS vs NOT IN	Error Handling	Hard